

MET Academy — Admin Portal

Phase 2 Developer Build Brief

v3 — comprehensive, gap-closed, developer-friendly

Prepared by: Ali Masood — Agentum AI

Client: METCARE® / MyEstheticTravel

Date: 26 May 2026

Companion documents: MET Academy Project Summary (May 2026); METCARE®
E-Learning Developer Fix Brief (T01–T30)

Executive summary

This brief defines Phase 2 of the MET Academy Admin Portal. The goal: give the METCARE® team (Mila, Philippe, Clara) and Agentum AI admins everything they need to run the platform without raising developer tickets.

In one paragraph

Phase 1 shipped a working baseline — Next.js admin portal, Supabase database, Cloudflare Stream video, EN/FR i18n, basic user CRUD, basic module CRUD. Phase 2 turns that into a full operational tool: it introduces the Formation hierarchy needed for future courses, adds proper video and quiz management, builds out user operations (search, filter, manual access grants, refunds), wires up the full payment dashboard with Stripe, and adds a structured CMS for every piece of editable copy on the public site. It also introduces two admin roles (Super Admin for Agentum, Content Manager for METCARE®), mandatory 2FA, an append-only audit log, and the governance plumbing that lets two teams use the same panel safely.

Scope at a glance

- 23 tickets across 6 parts (Foundation, Content, Users, Payments, Governance, System).
- Estimated effort: 8–12 dev weeks for one full-stack developer, less with parallel work on independent parts (Content and Payments can run in parallel after Foundation lands).
- Stack-locked to the existing Phase 1 setup: Next.js 16 + Supabase + Cloudflare Stream + Resend.
- Out of scope: GHL workflow configuration, Meta Pixel/GA4 setup, Stripe product creation (handled in Stripe dashboard), email template authoring.

The three highest-risk items

- T00 (Schema migration to introduce Formation hierarchy) — must complete before any other Phase 2 ticket. Run on a Supabase branch first.
- T05 (Per-video upload + 21-video bulk flow) — Vercel function size limits make direct browser-to-Cloudflare uploads mandatory, not a nice-to-have.
- T17 (Stripe payment dashboard) — coordinate with whoever builds the public-site Stripe Checkout. Both teams must agree on the webhook contract before either starts.

How to read this brief

Per-ticket structure

Every ticket follows the same eight-block format:

-  TL;DR — one-sentence summary so you know what the ticket is about at a glance.
-  Why this matters — concrete user-value reason this ticket exists.
-  Maps to — link back to the e-learning brief ticket(s) this depends on.
-  What exists today — honest snapshot of the Phase 1 state.
-  What needs to be built — plain-language description.
-  Required capabilities — feature list.
-  Codebase touch points — actual files/routes likely affected.
-  Important notes /  Acceptance criteria /  Manual test — gotchas, definition of done, click-by-click proof.

Status badges

- Type: ADD (net new), ENHANCE (extend existing), FIX (bug).
- Priority: HIGH (do first), MEDIUM (do soon), LOW (do later).
- Starting point: EXISTS (extend), PARTIAL (gaps to fill), NOT BUILT (greenfield).

Glossary — terms used throughout the brief

Term	Meaning
Formation	A course or programme. Top-level content unit. Examples: Initiation, Full Body, Massage Cicatriciel.
Module	A section inside a formation. Group of related videos + optional quiz.
Video	A single lesson video, hosted on Cloudflare Stream.
Tronc commun	Mandatory prerequisite formation (usually 'Initiation'). Some formations require it, some don't.
Initiation	The mandatory first-step formation. Always free, blocks access to Full Body until completed.
Full Body	The flagship paid formation. 21 videos. Requires Initiation completion.
Attestation de suivi	Completion proof PDF (NOT a certification — legally distinct, see T11).
Connected space	The logged-in area. Versus 'public space' which is the marketing site.
Pro guide	Paid-user-only library of audio podcasts + downloadable PDFs (per e-learning T15).
Super Admin	Agentum AI internal admin role. Full access including audit log, pricing, admin team.
Content Manager	METCARE® team admin role. Manages content, users, payments. No pricing, no audit log.

Term	Meaning
GHL	Go High Level — external CRM. Out of scope for Phase 2 except as a webhook target.
Resend	Transactional email service. Already wired in Phase 1 for activation + reset emails.

Two admin roles

- Super Admin — Agentum AI. Full access. Manages other admins, sees audit log, edits pricing, deletes data.
- Content Manager — METCARE® team. Manages content, users, payments. Cannot create admins, change pricing, or access the audit log.

Permissions are enforced server-side via middleware (T01). The UI hides what a role cannot do, but the server is the actual gate.

Phase 1 baseline — what already exists

Honest snapshot of the current Met_Academy_Admin_Portal as of May 2026. Phase 2 tickets are sized against this baseline.

What's built (don't rebuild this)

- Next.js 16 admin portal deployed on Vercel.
- Supabase PostgreSQL database with service_role access from admin routes.
- Users table with create / disable / activate flows. Single-query optimised routes.
- Modules table with create + thumbnail upload (race condition resolved).
- Cloudflare Stream integration for video hosting.
- i18n with locale files at src/i18n/locales/en.json and fr.json.
- Student-side enforcement: studentAuth middleware does live DB status check; deactivated users are kicked on next request via a global 401/403 interceptor.
- Resend wired for activation + password reset emails.

What's missing — the gap this brief closes

Capability area	Status today
Admin auth & roles	Single tier; no 2FA; no role separation.
Admin dashboard / landing page	Not built. Portal lands on Users tab.
Formation hierarchy	Not built. Modules are top-level.
Per-video metadata + bulk upload	Not built. Video appears tied to module at thumbnail level only.
Quiz builder	Not built.
Pro guide library (audio + PDFs)	Not built.
Testimonials manager	Not built.
Public-pages + dashboard CMS	Not built. All copy hard-coded.
Legal disclaimer + signature phrase	Not built.
Attestation PDF template	Not built.
User search/filter/detail	Users tab exists but limited to create + activate/disable.
Manual formation access grant/revoke	Not built. No formation-access entity.
Per-user progress override	Not built. No way to fix a stuck student.
Bulk user actions (CSV export)	Not built.
Payment dashboard (Stripe mirror)	Not built. No Stripe integration yet.
Refund flow with access revocation	Not built.

Capability area	Status today
Pricing & installments config	Not built.
Audit log	Not built.
Admin team management	Not built.
System settings (support, branding, banner)	Not built.



Common patterns (read once, reuse everywhere)

Several patterns recur across tickets. Documenting them once means each ticket can reference them without repeating itself. Implement these as shared helpers in Phase 2.

Pattern 1 — Role-protected route

Every admin API route must check the caller's role server-side. Build one middleware (e.g. `requireRole`) and use it everywhere:

Pattern

1. Read the admin session from the request (set by T01 auth middleware).
2. Compare against the route's required role.
3. If insufficient, return 403 with a clear error code (NOT a redirect — admins use direct API calls).
4. Capture the failed attempt in the audit log if it looks like privilege escalation (Content Manager hitting Super-Admin-only routes).

Pattern 2 — Audit log hook

Every mutation (create, update, delete) writes one `audit_log` row. Wrap mutation routes in a helper that captures before/after state automatically:

Pattern

1. Inside the route handler, after a successful mutation, call `await audit({admin, action_type, entity_type, entity_id, before, after})`.
2. The helper snapshots `admin.email` and `admin.role` at write time (so deleted admins still have intelligible log entries).
3. Do NOT log read operations (would explode the table size).
4. Audit log writes go to a separate database role with INSERT-only permission — no admin (not even Super Admin) can edit or delete log entries.

Pattern 3 — Archive, don't delete

Any entity tied to user history (formations, modules, videos, transactions) is archived, not deleted. Archived entities stay in the database but are hidden from new users.

Pattern

1. Every entity table has a status column with values like 'published' / 'archived' / 'deleted'.
2. 'archived' = hidden from public site but kept for historical records (user progress, attestations).
3. 'deleted' = soft delete with `deleted_at` timestamp (use sparingly, Super Admin only).
4. The UI shows 'Archive' as the default action; 'Delete' is hidden behind a confirmation modal.

Pattern 4 — Drag-and-drop reorder

Modules, videos, quiz questions, resources, testimonials all need user-controlled ordering. Build one reusable component + endpoint:

Pattern

1. Each row has a `display_order` integer field, contiguous per scope (e.g. `modules.display_order` is contiguous per `formation_id`).
2. Drag-and-drop in the UI; on drop, send a single batch reorder API call: `POST /api/admin/<entity>/reorder` with the new ordered array of IDs.
3. Server-side: update all rows in a single SQL transaction with their new `display_order` values.
4. Persist immediately — no separate 'Save order' button.

Pattern 5 — Bilingual content fields

Every user-facing string in content tables exists as a pair: `<field>_fr` and `<field>_en`. The public site reads FR; EN is for internal reference.

Pattern

1. Schema: `title_fr` (NOT NULL), `title_en` (nullable).
2. Admin UI: side-by-side FR and EN inputs, FR marked as required.
3. Public site fetches the FR version. The EN version exists for the admin team's internal documentation only.
4. Validation: if FR is empty, the row cannot be saved.

Pattern 6 — Public-site cache invalidation

When an admin edits content (CMS copy, formations, prices, testimonials), the public site must pick up changes within 60 seconds without a deploy.

Pattern

1. After a successful save, the admin route calls `POST` to `PUBLIC_SITE_REVALIDATE_URL` with a shared-secret token.
2. The public site exposes `/api/revalidate` that calls `Next.js revalidatePath/revalidateTag`.
3. Cache key strategy: tag-based, one tag per content area (homepage, signup, purchase, `formation:slug`, `testimonial-list`, etc.).
4. If the revalidate call fails, the admin sees a warning toast — content is saved but may take longer to appear publicly.



Build order & dependencies

Tickets are listed in the order they should be tackled. T00 is a hard prerequisite. After T00–T02, work can fan out — Content (Part B), Users (Part C), and Payments (Part D) are mostly independent and can be built in parallel by different developers.

Suggested sprint plan (single developer)

Sprint	Tickets	Theme
Sprint 1	T00, T01, T02	Foundation — schema migration, admin auth + 2FA, dashboard. Nothing else moves until this is done.
Sprint 2	T03, T04, T05	Content backbone — Formation manager, Module manager (extending Phase 1), Video upload.
Sprint 3	T06, T07, T08, T09	Content depth — Quiz builder, Pro guide, Testimonials, full CMS (public + dashboard copy).
Sprint 4	T10, T11	Legal + Attestation — disclaimer/signature manager, PDF template editor with 'certification' word-block.
Sprint 5	T12, T13, T14, T15, T16	User operations — list, detail, manual access, progress override, bulk export.
Sprint 6	T17, T18, T19	Payments — Stripe mirror dashboard, refund flow, pricing config.
Sprint 7	T20, T21, T22	Governance + system — audit log, admin team, system settings + banner.

Hard dependencies

- T00 blocks everything in Parts B–F. Nothing else starts until the Formation hierarchy migration is on production.
- T01 blocks T20 (audit log needs admin identity) and T22 (settings need role check).
- T04 (Module manager) depends on T00 (needs formation_id FK).
- T05 (Video manager) depends on T04 (videos belong to modules).
- T06 (Quiz) depends on T04 (quizzes attach to modules).
- T11 (Attestation) depends on T03 (formations have templates).
- T14 (Access grant) creates the user_formation_access table that T17 / T18 read for refund + revenue queries.
- T17 (Stripe) depends on the public-site Stripe Checkout being built in parallel.

Cross-reference: e-learning brief → admin brief

This table shows which admin ticket(s) implement each e-learning brief requirement. Use it to confirm full coverage and to trace 'why does this admin feature exist'.

E-learning #	Requirement	Admin ticket(s)
E-learning T01–T02	Two-tier + three-level access	T01 (admin auth separation) + T13/T14 (admin sees access level per user, can grant)
E-learning T04	Sign-up form fields	T13 (user detail surfaces all signup fields)
E-learning T06	Stripe payment + post-purchase	T17 (payment dashboard) + T22 (configurable notification recipient)
E-learning T07	Sequential unlock (Initiation → Full Body)	T03 (requires_initiation_id config) + T15 (per-user override)
E-learning T08	Free user dashboard layout	T09 (CMS for dashboard headers + block copy)
E-learning T09	Paid user dashboard layout	T09 (CMS for dashboard headers + block copy)
E-learning T10	Free modules section	T04 (modules can be marked free within a paid formation)
E-learning T11	Initiation container	T03 + T04 (formation + module structure)
E-learning T12	Full Body 21-video protocol	T05 (video manager + 21-video bulk upload)
E-learning T13	Quiz between Initiation modules	T06 (quiz builder)
E-learning T14	Attestation PDF auto-generation	T11 (PDF template editor with 'certification' word-block)
E-learning T15	Pro guide audio + PDFs	T07 (resource library)
E-learning T18–T24	Homepage, signup, purchase copy	T09 (public-pages CMS)
E-learning T25	Post-Initiation success message	T09 (milestone messages in CMS)
E-learning T26	End-of-journey completion message	T09 (milestone messages in CMS)
E-learning T27	Legal disclaimer (3 placements)	T10 (disclaimer manager)

E-learning #	Requirement	Admin ticket(s)
E-learning T28	Signature phrase (site-wide)	T10 (signature phrase manager)
E-learning T29	Future formations architecture	T00 (schema migration) + T03 (formation manager)
E-learning T30	Premium UX (locks, encouragement, support)	T09 (encouragement msgs in CMS) + T22 (support widget config)



Ticket index

#	Title	Type	Priority
00	Schema migration — introduce Formation hierarchy	ENHANCE	HIGH
01	Admin auth — Super Admin / Content Manager roles + 2FA	ENHANCE	HIGH
02	Admin dashboard — overview & quick actions	ADD	HIGH
03	Formation manager — list, create, edit, archive	ADD	HIGH
04	Module manager — extend with Formation grouping & quiz	ENHANCE	HIGH
05	Video manager — per-video CRUD + 21-video bulk upload	ADD	HIGH
06	Quiz builder — MCQ between Initiation modules	ADD	MEDIUM
07	Pro guide library — audio podcasts & PDF resources	ADD	MEDIUM
08	Testimonials manager	ADD	LOW
09	Full CMS — public pages, dashboards, milestone messages	ADD	HIGH
10	Legal disclaimer & signature phrase manager	ADD	MEDIUM
11	Attestation PDF template editor	ADD	MEDIUM
12	User list — search, filter, advanced view	ENHANCE	HIGH
13	User detail — progress, status, manual overrides	ADD	HIGH
14	Manual formation access grant / revoke	ADD	HIGH
15	Per-user progress override	ADD	MEDIUM
16	Bulk user actions — CSV export	ADD	LOW
17	Payment dashboard — Stripe mirror & transactions	ADD	HIGH
18	Refund flow with atomic access revocation	ADD	MEDIUM
19	Pricing & installments configuration	ADD	MEDIUM
20	Audit log — append-only event capture	ADD	HIGH
21	Admin team — invite, role change, deactivate	ADD	HIGH

#	Title	Type	Priority
22	System settings — support, branding, banner, notifications	ADD	MEDIUM



Part A — Foundation & access control

T00–T02 establish the data hierarchy and the admin entry point. T00 must land before any other Phase 2 ticket starts.

TICKET #00 — Schema migration — introduce Formation hierarchy

Type: **ENHANCE** | Priority: **HIGH** | Section: Database

Starting point: **PARTIAL — fill the gaps**



TL;DR

Add a 'formations' table on top of 'modules' so courses can be grouped properly. Backfill existing modules under a 'Full Body' formation.



Why this matters

Phase 1 has flat modules. The e-learning brief requires multiple formations (Full Body, Massage Cicatriciel, Lipœdème, IDE 360, etc.) that can share or skip the Initiation prerequisite. Without this, every new course is a code change.

Maps to: *E-learning brief T29 (architecture for future formations).*



What exists today

- modules table exists with title, thumbnail, and ordering.
- Cloudflare Stream video IDs are referenced per module.
- No formations table; modules are top-level.



What needs to be built

Create a formations table as the new top-level content entity. Each formation contains many modules. Each module contains many videos (see T05).

Add formation_id (FK) to the existing modules table. Backfill all existing modules into a default 'Full Body' formation.

Add requires_initiation_id (self-referential FK) on formations so courses can require Initiation as a prerequisite. Never hard-code 'Initiation' in app code.

Migration must run zero-downtime: add the column nullable, backfill, then add the NOT NULL constraint.



Required capabilities

- Table formations: id (uuid), slug, name_fr, name_en, description_fr, description_en, price_cents, currency, requires_initiation_id (uuid nullable, FK to formations.id), is_free (bool), formation_type (paid|free|mixed), status (draft|published|archived), display_order, created_at, updated_at.
- Add modules.formation_id (uuid, FK, NOT NULL after backfill).
- Seed: on first migration, create two formations — 'Initiation — Immersion' (free) and 'Full Body — Drainage post-opératoire' (paid). Backfill existing modules under Full Body.



Codebase touch points

- supabase/migrations/<timestamp>_formations.sql — the migration file.
- src/lib/db/types.ts (or equivalent) — TypeScript types for the new entity.



Important notes

- Test on a Supabase branch first. Confirm with PM that no student progress is lost.
- Document the rollback strategy in the migration file's header comments.

✓ Acceptance criteria

- Migration runs on production without downtime or data loss.
- All existing modules are visible inside the 'Full Body' formation after migration.
- Querying 'all modules in formation X' is a single indexed query.
- Existing student progress records still resolve correctly.

Manual test (how to prove it works)

On a Supabase branch: run the migration, log in as an existing test student, navigate to a module — it should load identically to before, but now nested under 'Full Body' in the admin formation tree.

TICKET #01 — Admin auth — Super Admin / Content Manager roles + 2FA

Type: **ENHANCE** | Priority: **HIGH** | Section: Auth

Starting point: **PARTIAL — fill the gaps**

TL;DR

Build a proper admin login at /admin/login with two roles and mandatory 2FA — completely separate from the student login.

Why this matters

Two teams will share this panel (Agentum + METCARE®). Without role separation, METCARE® content editors can accidentally change pricing or wipe data. Without 2FA, one compromised admin password = full platform takeover.

Maps to: *Brief-level requirement; supports e-learning T01 (two-tier access architecture).*

What exists today

- Admin authenticates but there is only one tier.
- No 2FA enforcement.
- No separate Super Admin vs Content Manager distinction.

What needs to be built

Create an admins table (separate from users — admins are NOT students).

Two roles: super_admin and content_manager.

Enforce TOTP 2FA on first login for all admins. No opt-out.

Hide UI elements server-side based on role. Never rely on client-side gating alone.

Session timeout: 60 minutes idle forces re-login.

Required capabilities

- Table admins: id, email (unique), name, password_hash, role, totp_secret (encrypted), totp_enrolled (bool), status (active|pending|deactivated), last_login_at, failed_login_count, locked_until, created_at.
- Email + password login at /admin/login. Reject any login attempt by an email that exists in the users table (clear separation).

- TOTP enrollment screen forced on first login. Show QR code + manual secret. Block dashboard access until enrolled.
- Brute-force protection: lock the admin account for 15 minutes after 5 failed attempts (logged in audit log per T20).
- Password reset email via Resend with a 1-hour expiry one-time link.
- Server-side role check on every admin route via shared middleware (e.g. `requireRole('super_admin')`).

Codebase touch points

- `src/app/api/admin/auth/login/route.ts` — login.
- `src/app/api/admin/auth/2fa/*` — TOTP enrollment + verification.
- `src/app/api/admin/auth/reset-password/*` — password reset flow.
- `src/middleware.ts` — admin route guard reading the role from session.
- `src/components/AdminLayout.tsx` — sidebar items conditionally rendered by role.
- `src/i18n/locales/en.json` + `fr.json` — translation keys for new auth flows.

Important notes

- Encrypt the TOTP secret at rest using a server-side key (env var `ADMIN_TOTP_ENC_KEY`, distinct from JWT secret).
- Issue a separate `ADMIN_SESSION_SECRET`; do not reuse `JWT_STUDENT_SECRET`.
- Build the force-logout endpoint here (used in T21).

Acceptance criteria

- Admin attempting to log in via `/sign-in` (student portal) is rejected.
- Student attempting to log in via `/admin/login` is rejected.
- First-login 2FA enrollment is mandatory and cannot be skipped.
- Content Manager hitting `/admin/audit-log` directly receives a 403.
- Sessions expire after 60 minutes idle.

Manual test (how to prove it works)

Create a Content Manager admin. Log in. Try to navigate to `/admin/audit-log` via URL bar — should see 403. Log out. Try to log in with that admin's email at `/sign-in` — should be rejected.

TICKET #02 — Admin dashboard — overview & quick actions

Type: **ADD** | Priority: **HIGH** | Section: Dashboard

Starting point: **NOT BUILT — net new**

TL;DR

Post-login landing page with platform stats, recent activity, and quick-action shortcuts.

Why this matters

The most common admin question is 'how is the platform doing right now?'. Without a dashboard, every admin opens the Users tab and starts hunting. This saves time on every single session.

What exists today

- Portal currently lands on the Users tab. No overview screen.

What needs to be built

Build /admin (root) as the post-login landing page.

Top stat band → activity feed → quick-action shortcuts → role-aware sidebar.

Required capabilities

- Stat band: total users, free users, paid users, users active today (last 24h based on last_login_at), revenue this month (from Stripe via T17), new signups today.
- Recent activity feed (last 20 events): new signup, new purchase, Initiation completed, Full Body completed, attestation generated. Each item links to the related user detail (T13).
- Quick actions panel: 'Upload videos', 'Create formation', 'Edit homepage copy', 'View payments', 'Manage users'.
- Sidebar nav with collapsible sections: Content, Users, Payments, Settings, Admin team. Last item is Super Admin only.
- Top-right: admin name + role badge + logout.

Codebase touch points

- src/app/admin/page.tsx — root admin page.
- src/components/AdminDashboard.tsx — composite dashboard component.
- src/app/api/admin/stats/route.ts — single endpoint returning all stat-band numbers.
- src/app/api/admin/activity/route.ts — paginated activity feed.

Acceptance criteria

- Dashboard loads in under 2 seconds on a cold cache.
- Stat figures reconcile with the underlying database (no caching surprises).
- Content Managers do not see the 'Admin team' sidebar item.
- Activity feed refreshes without page reload (poll every 30s or websocket — dev choice).

Manual test (how to prove it works)

Log in as Super Admin → see all sidebar sections including 'Admin team'. Log out, log in as Content Manager → 'Admin team' is missing. Create a test student signup on the public site → within 60s a 'new signup' event appears in the activity feed.

Part B — Content management

T03–T11 cover everything the METCARE® team needs to manage learning content, public-page copy, and the attestation PDF.

TICKET #03 — Formation manager — list, create, edit, archive

Type: **ADD** | Priority: **HIGH** | Section: Content / Formations

Starting point: **NOT BUILT — net new**

 **TL;DR**

CRUD interface for formations. Lets the team add Massage Cicatriciel, Lipœdème, IDE 360, etc. without dev help.

 **Why this matters**

Per the e-learning brief T29 the platform must support 10+ future formations without re-architecting. The Formation entity exists after T00; this is the UI to manage it.

Maps to: *E-learning brief T29.*

 **What exists today**

- No formations UI today. T00 creates the entity.

 **What needs to be built**

CRUD UI for formations.

List view sortable by name, type, price, created date, enrolled users count.

Detail view — edit properties, drill into modules (T04).

 **Required capabilities**

- Form fields: name (FR + EN), slug (auto-generated, editable), description (FR + EN), price (EUR), requires_initiation (dropdown of existing formations or 'None'), is_free (toggle), formation_type (paid|free|mixed), status (draft|published|archived), display_order.
- Per-formation actions: Edit, Duplicate (clone as template), Archive, View enrolled users.
- Archive (not delete) when enrolled users exist (see Common Pattern 3).
- Per-formation breadcrumb to its modules (T04 entry point).

 **Codebase touch points**

- src/app/admin/formations/page.tsx — list view.
- src/app/admin/formations/[id]/page.tsx — detail / edit view.
- src/app/api/admin/formations/route.ts — list + create.
- src/app/api/admin/formations/[id]/route.ts — get/update/archive.

 **Important notes**

- Future formations the system must support (per e-learning brief T29): Massage Cicatriciel, Full Face, Drainage lymphatique & pré-opératoire, Lipœdème, IDE 360 (no tronc commun), Aesthetic Planner Advisor, Marketing Digital, Ads META, Instagram.
- On Publish, trigger public-site revalidation (Common Pattern 6).

 **Acceptance criteria**

- Creating a new formation end-to-end requires zero code changes.
- Toggling `requires_initiation` on a published formation updates the unlock cascade for affected users.
- Archiving removes the formation from the public site without breaking existing users' progress.



Manual test (how to prove it works)

Create a test formation 'Massage Cicatriciel', set price €199, set `requires_initiation` = Initiation. Publish. Check the public site — it appears in the formation catalogue. Archive it. Check the public site — it's gone, but a test student who had purchased it still sees their progress.

TICKET #04 — Module manager — extend with Formation grouping & quiz attachment

Type: **ENHANCE** | Priority: **HIGH** | Section: Content / Modules

Starting point: **EXISTS — extend it**



TL;DR

Extend the existing Phase 1 modules tab to nest modules under formations and attach optional quizzes.



Why this matters

Modules need to live inside formations (T00). They also need to optionally gate completion on a quiz (e-learning T13).

Maps to: E-learning brief T11, T12, T13.



What exists today

- Modules table and module-creation flow exist (Phase 1).
- Thumbnail upload flow exists and is stable after the race-condition fix.
- Module-creation API is single-query optimized.
- Modules are flat — no Formation parent and no Video children today.



What needs to be built

Extend the existing module manager: scope modules under a Formation (T00), reorder via drag-and-drop within a formation (Common Pattern 4), attach an optional quiz (T06), link videos as children (T05).

List view becomes: Formation → Modules tree. Click a module to drill into its Videos list.



Required capabilities

- New module fields: `formation_id` (FK), `description` (FR + EN), `completion_required` (bool — if true, all child videos must be complete), `has_quiz` (bool), `quiz_id` (FK nullable), `is_free` (bool — for marking specific modules as free even within a paid formation, per e-learning T10).
- Drag-and-drop reorder of modules within a formation.
- Drag-and-drop reorder of videos within a module.
- Module-level preview link — open the user-facing version in a new tab.
- Archive (not delete) for modules with enrolled in-progress users.

📁 Codebase touch points

- src/components/ModulesTab.tsx — refactor for nested formation grouping.
- src/app/api/admin/modules/route.ts — extend query to filter by formation_id.
- src/app/api/admin/modules/[id]/route.ts — add fields.
- src/app/api/admin/modules/reorder/route.ts — new batch-reorder endpoint.

✅ Acceptance criteria

- Existing modules continue to render after migration with no regression.
- Drag-and-drop reorder persists without a separate save action.
- Renaming a module reflects on the live site within 60 seconds.
- Existing thumbnail upload flow continues to work unchanged.
- is_free toggle on a module within a paid formation makes that module accessible to free users.

🔧 Manual test (how to prove it works)

Create a module 'Présentation de l'initiation' inside Full Body, mark is_free=true. Log in as a free user on the public site — that module is playable; other Full Body modules are locked.

TICKET #05 — Video manager — per-video CRUD + 21-video bulk upload

Type: **ADD** | Priority: **HIGH** | Section: Content / Videos

Starting point: **PARTIAL — fill the gaps**

📌 TL;DR

Build a proper videos table with per-video metadata and a 21-video bulk-upload flow direct to Cloudflare Stream.

💡 Why this matters

Full Body has 21 videos (e-learning T12). Uploading them one at a time through a Vercel function would hit the 4.5MB body limit immediately. Direct browser → Cloudflare uploads are mandatory.

Maps to: *E-learning brief T12.*

📌 What exists today

- Cloudflare Stream is integrated (CF_ACCOUNT_ID, CF_STREAM_API_TOKEN configured).
- Modules have thumbnails uploaded via the admin portal.
- No per-video entity — module currently references a single Cloudflare Stream video at module level.

🧱 What needs to be built

Introduce a videos table as a child of modules. A module can have many videos with order and metadata.

Per-video upload through the admin portal — directly to Cloudflare Stream.

Bulk upload: drag 21 files at once, assign to a module in sequence.

Replace video file without losing metadata or user progress.

Required capabilities

- Table videos: id, module_id (FK), title_fr, title_en, description_fr, description_en, cloudflare_stream_id, duration_seconds, thumbnail_url, display_order, status (draft|published), created_at.
- Upload UI: direct browser-to-Cloudflare upload (tus protocol or signed upload URLs) — large files bypass Vercel.
- Progress bar that survives tab close — show 'resume upload' on return.
- Bulk upload: drag multiple files, assign module + display_order to all at once.
- Replace: keep videos.id, swap cloudflare_stream_id, preserve progress records.

Codebase touch points

- supabase/migrations/<timestamp>_videos.sql — videos table.
- src/app/admin/modules/[id]/videos/page.tsx — per-module video list.
- src/app/api/admin/videos/upload-url/route.ts — issue Cloudflare Stream signed URLs.
- src/app/api/admin/videos/route.ts — CRUD.
- src/app/api/admin/videos/bulk/route.ts — bulk metadata assignment after upload.

Important notes

- Vercel serverless functions cap at 4.5MB request body. Proxying uploads through the API is not an option. Direct browser-to-Cloudflare is mandatory.
- Auto-generated thumbnails come from Cloudflare Stream's API; allow manual override via the existing thumbnail flow.
- If the current module-level video reference is in use, write a migration that converts each module's single video into a videos row with display_order=1.

Acceptance criteria

- A 2GB video upload completes successfully and is playable on the public site within 5 minutes.
- Replacing a video file preserves all user completion records.
- Bulk upload of 21 videos completes in a single admin session and assigns them to the Full Body module in correct order.
- Upload failures surface a clear error message, not a stack trace.

Manual test (how to prove it works)

Drag 21 test .mp4 files into the bulk uploader on the Full Body module. All 21 reach 'published' status within 10 minutes. Log in as a test paid user — all 21 are listed in order, all play.

TICKET #06 — Quiz builder — MCQ between Initiation modules

Type: **ADD** | Priority: **MEDIUM** | Section: Content / Quizzes

Starting point: **NOT BUILT — net new**

TL;DR

Multiple-choice quiz builder. The METCARE® team writes questions later — this is the UI to do it without dev help.

Why this matters

E-learning T13 requires a quiz between Initiation modules. The questions aren't ready yet, so the panel must let the team add them later.

Maps to: *E-learning brief T13*.



What exists today

- No quiz tables, no quiz UI.



What needs to be built

Tables: quizzes, questions, answers, quiz_attempts.

Per-quiz settings + per-question CRUD + drag-reorder.

Preview the quiz exactly as a user will see it.



Required capabilities

- Per-quiz: title (FR + EN), pass threshold (% correct required, default 70%), max attempts (default unlimited), shuffle questions (bool), shuffle answers (bool).
- Per-question: text (FR + EN), 2–6 answer options, mark correct options, optional explanation shown after submission.
- Drag-to-reorder questions within a quiz (Common Pattern 4).
- Attach quiz to a module via modules.quiz_id (T04). Module completion gated on quiz pass.
- Active/inactive toggle so a quiz can be disabled without deletion.



Codebase touch points

- supabase/migrations/<timestamp>_quizzes.sql.
- src/app/admin/quizzes/* — full CRUD UI.
- src/app/api/admin/quizzes/* — endpoints.



Acceptance criteria

- Adding a new question and saving updates the live quiz immediately.
- Changing the pass threshold from 70% to 80% does NOT retroactively un-complete users who already passed at 70%.
- Admin can disable a quiz on a module without deleting it.



Manual test (how to prove it works)

Create a 3-question quiz, threshold 70%, attach to an Initiation module. Log in as a test student, take the quiz, fail it with 33%. See retry option. Pass with 100%. The Initiation module marks complete; Full Body unlocks.

TICKET #07 — Pro guide library — audio podcasts & PDF resources

Type: **ADD** | Priority: **MEDIUM** | Section: Content / Resources

Starting point: **NOT BUILT — net new**



TL;DR

Upload audio podcasts (.mp3, .m4a) and downloadable PDFs for the paid-user-only Pro guide section.

Why this matters

E-learning T15 calls for an 'Espace guide pro audio podcast + téléchargement'. METCARE® team must manage it themselves.

Maps to: *E-learning brief T15*.

What exists today

- Supabase storage bucket THUMBNAIL_BUCKET exists.
- No resources entity, no audio or PDF upload flow.

What needs to be built

Two resource types: audio (streamable + downloadable) and document (downloadable PDF).

Upload UI backed by Supabase storage.

Replace file without breaking existing bookmarks (slug stable).

Required capabilities

- Table resources: id, type (audio|document), title_fr, title_en, description_fr, description_en, file_url (Supabase storage), thumbnail_url, access_formation_id (FK — which formation grants access), display_order, status (published|hidden), download_count, play_count.
- Upload for audio (.mp3, .m4a) and documents (.pdf).
- New Supabase storage buckets: 'resources-audio' and 'resources-docs'.
- Track download/play counts per resource (read-only stat).

Codebase touch points

- supabase/migrations/<timestamp>_resources.sql + storage bucket creation.
- src/app/admin/resources/* — CRUD UI.
- src/app/api/admin/resources/* — endpoints.

Acceptance criteria

- Admin uploads a new audio podcast and it appears in paid users' Pro guide section within 1 minute.
- Replacing a PDF preserves the existing URL slug.
- Audio streams on both desktop and mobile.

Manual test (how to prove it works)

Upload a 50MB .mp3 podcast. Log in as a test paid user — Pro guide section shows the podcast, audio plays in-browser, download button works. Replace the .mp3 with a new file. Same URL still works.

TICKET #08 — Testimonials manager

Type: **ADD** | Priority: **LOW** | Section: Content / Testimonials

Starting point: **NOT BUILT — net new**

TL;DR

Add, edit, reorder, and hide testimonials shown on the free user dashboard.

Why this matters

E-learning T08 lists 'Témoignages' as a required block on the free dashboard. Today these are hard-coded.

Maps to: *E-learning brief T08.*

What exists today

- Testimonials are hard-coded on the public site today.

What needs to be built

CRUD for testimonials with drag-reorder (Common Pattern 4) and bilingual fields (Pattern 5).

Required capabilities

- Table testimonials: id, author_name, author_role, country, photo_url, quote_fr, quote_en, rating, display_order, status, tagged_formation_id (nullable for filtering).
- Toggle visibility without deleting.
- Optional tag by formation for formation-relevant testimonials on each formation page.

Acceptance criteria

- Adding a testimonial publishes to the free dashboard within 30 seconds.
- Reordering reflects on the live site without re-deploy.

Manual test (how to prove it works)

Add a testimonial with photo upload, FR + EN quotes. Set display_order = 1. Open the public-site free dashboard — that testimonial appears first.

TICKET #09 — Full CMS — public pages, dashboards, milestone messages

Type: **ADD** | Priority: **HIGH** | Section: Content / CMS

Starting point: **NOT BUILT — net new**

TL;DR

Structured CMS covering every editable string on the public site: homepage, signup, purchase, dashboard headers, lock screens, milestone messages, and encouragement messages.

Why this matters

The e-learning brief specifies exact French copy for 12+ pages and states. None of it is editable today — every typo, tone tweak, or copy change is a developer ticket. This is the single biggest 'no-dev-help' unlock for the METCARE® team.

Maps to: *E-learning brief T06, T07, T08, T09, T18, T19, T20, T21, T22, T23, T24, T25, T26, T30.*

What exists today

- All copy is hard-coded in the public-site repo.

What needs to be built

Build a STRUCTURED CMS for every page/state defined in the e-learning brief. Each editable field is a known slot with a known label — not a free-form page builder.

Coverage areas (don't miss any):

Required capabilities

- PUBLIC PAGES (e-learning T18–T24): homepage hero (title, subtitle, body, 2 CTAs, 4 social-proof numbers), homepage sections 1–4 (title + body + list items), signup page (title + intro), purchase page (hero title + body + 5-item 'what you get' list + 4-verb positioning block + CTA).
- DASHBOARDS (e-learning T08, T09): free user dashboard header + Block 1/2/3 titles/bodies/CTAs/lock message; paid user dashboard header + Step 1/Step 2 copy + attestation block copy.
- FREE MODULES SECTION (e-learning T10): section name + persistent CTA.
- LOCK SCREENS (e-learning T07): the 'Votre parcours commence par l'initiation...' message + button label.
- POST-PURCHASE REDIRECT (e-learning T06): 'Bienvenue dans votre parcours Full Body METCARE@...' welcome message.
- MILESTONE MESSAGES (e-learning T25, T26): post-Initiation success message + end-of-journey completion message.
- ENCOURAGEMENT MESSAGES (e-learning T30): editable list of post-module-completion messages like 'Bien joué!', 'Vous avancez bien!', 'Bravo, vous progressez avec constance.'
- TECHNICAL: tables `page_content` (`page_key`, `field_key`, `value_fr`, `value_en`, `updated_by`, `updated_at`) + `page_content_revision` (last 10 versions per field). All fields FR-first with EN reference column. Rich text for body fields (bold/italic/line breaks only, no free HTML). Preview button shows rendered page with proposed changes. Publish vs Save Draft. Glossary panel with approved phrases ('drainage post-opérateur à visée esthétique').

Codebase touch points

- `supabase/migrations/<timestamp>_cms.sql` — `page_content` + `page_content_revision` tables.
- `src/app/admin/cms/*` — structured editor UI organized by page area.
- Public site repo — replace hard-coded copy with fetches from `page_content` via a cached API.
- Public site `/api/revalidate` — endpoint Phase 2 calls on Publish (Common Pattern 6).

Important notes

- DO NOT build a 'create new page' feature. Only the pages/states defined in the e-learning brief are editable.
- Coordinate with the public-site team to define the API contract for fetching `page_content`.
- Encouragement messages should be a LIST that the team can add to over time, not a fixed N slots. The frontend randomly picks one on each module completion.

Acceptance criteria

- Editing the homepage hero title and clicking Publish reflects on the live site within 60 seconds.
- Editing the free dashboard Block 3 lock message reflects on the dashboard within 60 seconds.
- Preview shows the actual styled page, not a plain-text dump.
- Reverting to a previous revision restores all fields atomically.
- Saving a draft does not affect the live site.

Manual test (how to prove it works)

Change the homepage hero title to 'Test 26 May'. Click Publish. Within 60s, refresh the public homepage — new title is live. Revert to previous revision. Refresh — original title is back.

TICKET #10 — Legal disclaimer & signature phrase manager

Type: **ADD** | Priority: **MEDIUM** | Section: Content / CMS

Starting point: **NOT BUILT — net new**

TL;DR

Edit the legal disclaimer once; it updates in all three placements (purchase page, footer, attestation screen). Same for the signature phrase.

Why this matters

E-learning T27 requires the same legal disclaimer in 3 places. T28 requires the signature phrase in 2–3 spots. If they're hard-coded, every edit risks divergence — and the disclaimer is legally sensitive.

Maps to: *E-learning brief T27, T28.*

What exists today

- Both are currently hard-coded on the public site if present at all.

What needs to be built

Single source of truth — edit once, propagates everywhere.

Per-content placement toggles.

Required capabilities

- Legal disclaimer: editable FR text, placement toggles (purchase_page, footer, attestation_screen — all three on by default).
- Signature phrase: editable FR text, placement toggles (footer, purchase_page_above_fold, transactional_emails).
- Length validation warning if disclaimer exceeds allotted space on the attestation screen.
- Lock toggle on the disclaimer — Super Admin can lock so Content Managers cannot edit (legal-sensitive).

Acceptance criteria

- Updating the disclaimer once updates all three placements simultaneously.
- When locked, the field is read-only to Content Managers.
- Disabling a placement removes the disclaimer from that location within 1 minute.

Manual test (how to prove it works)

As Super Admin, edit the disclaimer text and lock it. Switch to a Content Manager session — the disclaimer field is read-only. Disable the footer placement. Refresh the public site footer — disclaimer no longer appears there but still appears on the purchase page.

TICKET #11 — Attestation PDF template editor

Type: **ADD** | Priority: **MEDIUM** | Section: Content / Attestation

Starting point: **NOT BUILT — net new**

 **TL;DR**

Edit the attestation PDF heading, body, logo, signature — field by field, no PDF designer. Server blocks the word 'certification'.

 **Why this matters**

E-learning T14 makes this legally critical: the PDF must say 'attestation de suivi', never 'certification'. A typo or admin error here creates real liability. Hard-coding the validation server-side removes the risk.

Maps to: *E-learning brief T14*.

 **What exists today**

- No attestation generation flow exists yet.

 **What needs to be built**

Field-by-field editor with predefined slots — no free-form PDF designer.

Per-formation override (each formation can have its own template).

Server-side validation that blocks 'certification' / 'certifié' / 'certifie' (case-insensitive).

 **Required capabilities**

- Table `attestation_templates`: `id`, `formation_id` (FK), `heading_fr`, `body_fr` (with token placeholders), `logo_url`, `signature_image_url`, `signature_name`, `updated_at`.
- Body supports tokens: `{first_name}`, `{last_name}`, `{formation_name}`, `{completion_date}`.
- Sample PDF preview with dummy data ('Jean Dupont', '15/01/2026').
- PDF library: server-side, dev choice (recommend `@react-pdf/renderer` or `pdfkit`).

 **Codebase touch points**

- `supabase/migrations/<timestamp>_attestation_templates.sql`.
- `src/app/admin/attestation/*` — UI.
- `src/app/api/admin/attestation/route.ts` — with the validator middleware that rejects forbidden words.
- `src/lib/pdf/attestation.ts` — PDF generation.

 **Important notes**

- ENFORCE: the words 'certification', 'certifié', 'certifie' (case-insensitive) must be rejected by server-side validation on `heading_fr` and `body_fr`. Legally required per e-learning T14.
- 'Attestation de suivi' wording must remain.
- Existing issued attestations stay as-issued (immutable). Only NEW attestations use the updated template.
- Recommended PDF title text (FR): 'Attestation de suivi — Parcours Full Body METCARE®' (e-learning T14).

 **Acceptance criteria**

- Editing the heading text and saving regenerates all NEW attestations with the new text.
- Attempting to save a template containing 'certification' returns a validation error referencing the legal restriction.

- Sample preview matches the production PDF byte-for-byte (apart from dummy data).



Manual test (how to prove it works)

Edit the heading to 'Attestation de certification'. Click Save. Server returns a clear error: 'Forbidden word: certification. Use "attestation de suivi" only.'

Part C — User management

T12–T16 extend the existing Phase 1 users tab into a complete user operations surface.

TICKET #12 — User list — search, filter, advanced view

Type: **ENHANCE** | Priority: **HIGH** | Section: Users / List

Starting point: **PARTIAL — fill the gaps**

TL;DR

Extend the existing users tab with search, filters, and progress columns so the team can answer 'has Marie Dupont completed her Initiation?' in seconds.

Why this matters

Today the users tab is a flat list. The METCARE® team handles customer questions daily — without proper search and filtering, every support ticket becomes a manual Supabase query.

What exists today

- Users tab exists with create / disable / activate actions.
- UsersTab.tsx has DOM stability fix applied.
- i18n keys wired for existing actions.

What needs to be built

Extend the existing users tab into a full search + filter + advanced view.

Default view: paginated table sorted by signup date (newest first).

Required capabilities

- Search: name, email, phone, country.
- Filters: access level (no purchase / free / paid by formation), Initiation status, Full Body status, signup date range, country, profession, main objective.
- Columns: name, email, access level, signup date, last active, Initiation status, Full Body status, attestation issued.
- Click a row → user detail (T13).
- Result count visible: 'Showing 1–50 of 1,247 users'.
- URL reflects filter state so admins can share filtered views.

Codebase touch points

- src/components/UsersTab.tsx — extend with filter sidebar + advanced search.
- src/app/api/admin/users/route.ts — add filter/sort query params + DB-side filtering.

Important notes

- Index on users.email, users.signup_date, and user_formation_access columns (added in T14) is required for the under-1-second target.
- Don't query progress per row in a loop — join on user_formation_access in the main query.

Acceptance criteria

- Search by name or email returns results in under 1 second across at least 10,000 records.

- Filters can be combined (e.g. 'paid Full Body + Initiation completed + France').
- URL reflects current filter set.

Manual test (how to prove it works)

Filter by 'paid Full Body + France + Initiation not started'. URL updates. Copy URL, paste in a new tab — same filtered view loads.

TICKET #13 — User detail — progress, status, manual overrides

Type: **ADD** | Priority: **HIGH** | Section: Users / Detail

Starting point: **NOT BUILT — net new**

TL;DR

Single page showing everything about one user: profile, full progress, history. Plus action buttons for manual overrides.

Why this matters

Customer support calls require admins to see what a student has done and intervene. Without this page, every support call is a multi-tab database lookup.

What exists today

- No dedicated user detail page.

What needs to be built

Single-page view of one user. Three tabs: Profile / Progress / History.

Action buttons in the page header for common overrides.

Required capabilities

- PROFILE tab: first name, last name, email (editable), phone, country, profession, main objective, signup date, last login, account status. All editable, captured in audit log (T20).
- PROGRESS tab: per formation the user has access to, list modules with per-video completion checkmarks, time spent, last activity, current step. Visual progress bars per formation.
- HISTORY tab: chronological feed — signed up, purchased Full Body (€X), started Initiation, completed Initiation, etc. Each timestamped.
- Action buttons (top right): Grant access (links to T14), Reset password (Resend pipe), Suspend account, Resend welcome email, Regenerate attestation PDF, GDPR export, Force re-sync to GHL, Delete user (Super Admin only).
- Force re-sync to GHL = read-only call to the existing webhook with the user's current state. GHL config out of scope.
- Delete requires typing the user's email to confirm.

Codebase touch points

- `src/app/admin/users/[id]/page.tsx` — new detail page.
- `src/app/api/admin/users/[id]/route.ts` — extend with GET + PATCH.
- `src/app/api/admin/users/[id]/progress/route.ts` — progress aggregation.
- `src/app/api/admin/users/[id]/history/route.ts` — chronological event feed.
- `src/app/api/admin/users/[id]/gdpr-export/route.ts` — JSON export.

✓ Acceptance criteria

- Profile changes are logged in audit log with before/after values.
- Resetting password sends an email and invalidates existing sessions (use existing student session invalidation).
- GDPR export downloads a JSON file with all user data within 30 seconds.
- Delete (Super Admin only) requires typing the user's email.



Manual test (how to prove it works)

Open a test student's detail page. Edit their phone number. Save. Open T20 audit log — entry exists showing the phone change with before/after values.

TICKET #14 — Manual formation access grant / revoke

Type: **ADD** | Priority: **HIGH** | Section: Users / Access

Starting point: **NOT BUILT — net new**



TL;DR

Admins can grant or revoke formation access manually. VIP partners, refunds, B2B deals, comped accounts.



Why this matters

Real-world ops needs this constantly. Without it, every comped account = a developer running a manual DB UPDATE. Also the foundation for T18 (refund flow auto-revokes access).



What exists today

- No `formation_access` entity. Access is implicit and not modeled.



What needs to be built

Create `user_formation_access` — the single source of truth for who has access to what.

Admins can grant access manually (VIP, refunds, comped, B2B paid outside Stripe).

Reverse: revoke access (chargeback, fraud).



Required capabilities

- Table `user_formation_access`: `id`, `user_id` (FK), `formation_id` (FK), `source` (`purchase|manual_grant|promo`), `granted_by_admin_id` (nullable), `reason` (required free text), `stripe_charge_id` (nullable), `granted_at`, `revoked_at` (nullable), `revoked_by_admin_id` (nullable), `revoke_reason`.
- Grant flow from user detail (T13): button → dropdown of published formations → required reason → grant.
- Granted access activates immediately and follows the same unlock cascade as a paid purchase (Initiation unlocks → Full Body unlocks after Initiation completes — per e-learning T07).
- Revoke flow: button on existing access row → required reason → revoke. Does NOT delete progress. If re-granted, progress resumes.
- Bulk grant: upload CSV of emails + target formation in one operation.



Codebase touch points

- supabase/migrations/<timestamp>_user_formation_access.sql.
- src/app/api/admin/users/[id]/access/route.ts — grant/revoke.
- src/app/api/admin/users/access/bulk/route.ts — CSV bulk grant.
- user-portal-backend — update access checks to query user_formation_access (replacing whatever implicit check exists today).

✓ Acceptance criteria

- Granting access activates the formation within 30 seconds.
- Manually granted users do NOT appear in revenue reports as paid customers (filter by source).
- Revoking access removes ability but preserves progress.
- Every grant and revoke captured in the audit log.
- Bulk grant of 50 users completes in under 2 minutes.

🔧 Manual test (how to prove it works)

Grant Full Body access to a test free user with reason 'VIP partner Q2 2026'. Log in as that user — Full Body is unlocked, Initiation is the required first step. Revoke access with reason 'chargeback'. User loses access on next request; their existing progress on Initiation persists in DB.

TICKET #15 — Per-user progress override

Type: **ADD** | Priority: **MEDIUM** | Section: Users / Progress

Starting point: **NOT BUILT — net new**

📌 TL;DR

Mark a specific video or module complete/incomplete for one user. Reset their progress. Force-unlock the next step.

💡 Why this matters

Real support cases: a student says 'I watched video 7 but it's not marked complete' (Cloudflare Stream glitch). Or a student finished Initiation elsewhere and needs Full Body unlocked. Without this, every case is escalated to a developer.

Maps to: E-learning brief T07 (unlock cascade — this is the admin override for it).

📌 What exists today

- No progress mutation tooling exists.

🧱 What needs to be built

On user detail PROGRESS tab (T13): per-video and per-module override actions.

Three actions: Mark complete, Mark incomplete, Reset progress.

🔧 Required capabilities

- Per-video row: button 'Mark complete' / 'Mark incomplete' (toggle). Records completed_at = now, completed_by_admin_id, override_reason (required).
- Per-module row: button 'Mark module complete' (marks all child videos + quiz attempt passed).
- Per-formation row: button 'Reset all progress' with hard confirmation (typed user email).

- Force-unlock next step: button bypasses the unlock cascade for this user only — e.g. unlock Full Body without requiring Initiation completion.
- All overrides logged in audit log with reason.

Codebase touch points

- `src/app/api/admin/users/[id]/progress/override/route.ts` — mutation endpoint.
- `src/app/admin/users/[id]/page.tsx` — UI on PROGRESS tab.

Important notes

- Force-unlock should be visually distinct from normal completion in the user-facing dashboard? Or transparent? Decision needed from PM. Default: transparent (the user sees normal unlock state, override is internal).
- Reset progress is destructive — Super Admin only, double-confirm required.

Acceptance criteria

- Marking a video complete for one user updates that user's progress within 5 seconds without affecting other users.
- Force-unlock Full Body for a user lets them access Full Body videos even if Initiation is not marked complete.
- Every override is captured in the audit log with the required reason.
- Reset progress requires Super Admin + typed email confirmation.

Manual test (how to prove it works)

Pick a test paid user whose Initiation is incomplete. Force-unlock Full Body with reason 'completed elsewhere'. Log in as that user — Full Body videos are playable. Audit log shows the override with the reason.

TICKET #16 — Bulk user actions — CSV export

Type: **ADD** | Priority: **LOW** | Section: Users / Bulk

Starting point: **NOT BUILT — net new**

 **TL;DR**

Export the current filtered user list to CSV. Optional: push a segment to GHL as a tag.

Why this matters

Most workflows go through GHL but the team will sometimes need ad-hoc analysis (revenue cohort, country breakdown). CSV export inside the panel saves a dev request.

What exists today

- No export functionality.

What needs to be built

From a filtered user list (T12): button 'Export to CSV' downloads the current filter result. Optional 'Send to GHL' button — read-only call to the existing webhook.

Required capabilities

- Configurable column selection before export.
- Hard cap: 50,000 rows per export.

- CSV format: UTF-8 with BOM so French characters render correctly in Excel.

✓ **Acceptance criteria**

- CSV export of 1,000 users completes in under 10 seconds.
- Exported CSV opens cleanly in Excel and Google Sheets with French characters intact.

🔧 **Manual test (how to prove it works)**

Filter by country=France. Export to CSV. Open in Excel — French characters (é, à, ç) render correctly, not as garbled bytes.

Part D — Payment operations

T17–T19 add payment operations. Stripe is the source of truth — this panel mirrors transactions for context inside the admin workflow.

TICKET #17 — Payment dashboard — Stripe mirror & transactions

Type: **ADD** | Priority: **HIGH** | Section: Payments / Dashboard

Starting point: **NOT BUILT — net new**

 **TL;DR**

Read-only operational dashboard mirroring Stripe transactions. Admins shouldn't need to open Stripe directly for daily work.

 **Why this matters**

Stripe dashboard is fine for accountants but slow for ops. Mirroring inside the admin panel + linking transactions to user records makes refunds, customer support, and revenue checks one-click instead of cross-tab.

Maps to: *E-learning brief T06.*

 **What exists today**

- No Stripe integration in the admin portal.
- Per e-learning T06, Stripe is planned but implementation hasn't started.

 **What needs to be built**

Read-only dashboard with revenue stats, transactions table, filters.

Stripe webhook receiver populates a mirror table.

 **Required capabilities**

- Revenue stats (top band): revenue today / this week / this month / this year, average order value, refund rate. Currency selector (default EUR).
- Transactions table: date, customer (clickable to T13), formation, amount, currency, status, payment method, Stripe txn ID (copy button).
- Filters: date range, status, formation, payment method.
- Export filtered transactions to CSV.
- Click transaction → drawer with full Stripe metadata (read-only).

 **Codebase touch points**

- supabase/migrations/<timestamp>_transactions.sql — mirror table.
- src/app/admin/payments/page.tsx — dashboard UI.
- src/app/api/webhooks/stripe/route.ts — webhook receiver.
- src/app/api/admin/payments/* — read endpoints.

 **Important notes**

- Stripe is SOURCE OF TRUTH. Mirror table is populated by webhooks. If they disagree, Stripe wins — surface a warning in audit log.
- This view MUST NOT support editing transaction data. Refunds go through T18.

- Coordinate with the e-learning public-site team: the public Stripe Checkout flow must be built in parallel. Agree on webhook event types before either side starts.

✓ Acceptance criteria

- Today's revenue figure matches Stripe Dashboard within €1.
- A new successful payment appears in the table within 60 seconds via webhook.
- Refunded transactions show a clear badge.
- Date range picker defaults to last 30 days.

Manual test (how to prove it works)

Make a test purchase via Stripe test mode on the public site. Within 60s, the transaction appears in the admin payment dashboard with the correct amount + customer email.

TICKET #18 — Refund flow with atomic access revocation

Type: **ADD** | Priority: **MEDIUM** | Section: Payments / Refunds

Starting point: **NOT BUILT — net new**

TL;DR

Issue a refund AND revoke formation access in one operation. Permission-gated by amount.

Why this matters

Refunding via Stripe then manually revoking in two steps is error-prone — admins forget the revoke. Atomic operation eliminates the gap. Permission cap protects against rogue large refunds.

What exists today

- No refund mechanism. Today's refunds happen directly in Stripe dashboard.

What needs to be built

Issue refund + remove formation access in one operation with full audit trail.

Two types: full refund (revoke access) and partial refund (keep access — goodwill gesture).

Required capabilities

- From transaction row or user detail: 'Issue refund' modal with amount (default full), required reason, 'also revoke formation access' checkbox (default ON for full refunds).
- Calls Stripe refund API server-side. On success, marks transaction refunded and (if checkbox on) revokes formation access via T14's mechanism.
- Atomic: refund failure does NOT revoke access (must succeed or fail cleanly).
- Email confirmation to customer via existing Resend pipe.
- Permission gate: Content Managers can refund up to €500. Refunds over €500 require Super Admin approval ('Awaiting Super Admin approval' state).

Codebase touch points

- `src/app/api/admin/payments/[id]/refund/route.ts` — refund endpoint with transaction wrapping.
- `src/app/api/admin/payments/[id]/approve-refund/route.ts` — Super Admin approval endpoint.

✓ Acceptance criteria

- Full refund completes in under 10 seconds; customer loses access immediately.
- Refund record appears in T17 dashboard within 60 seconds.
- Audit log captures who, when, amount, reason.
- Content Manager attempting €600 refund sees approval-required state, not an error.

Manual test (how to prove it works)

As Content Manager, attempt to refund a €500 transaction with reason 'customer requested'. Refund succeeds, access revoked. Now attempt to refund a €700 transaction — see 'Awaiting Super Admin approval' state. Log in as Super Admin → approve → refund completes.

TICKET #19 — Pricing & installments configuration

Type: **ADD** | Priority: **MEDIUM** | Section: Payments / Pricing

Starting point: **NOT BUILT — net new**

TL;DR

Edit formation prices, toggle installment payments, manage promo codes — all syncing to Stripe via API.

Why this matters

E-learning T06 requires the platform to support 'paiement en plusieurs fois' without being blocked at launch. This is where the team configures it when ready, plus everyday price changes.

Maps to: *E-learning brief T06*.

What exists today

- Pricing is hard-coded if present at all.

What needs to be built

Per-formation pricing config.

Promo codes (basic).

Permission: Super Admin only. Content Managers see pricing as read-only.

Required capabilities

- Per formation: base price (EUR), currency, VAT inclusive (bool).
- Installments per formation: toggle, number of installments (2|3|4), frequency (monthly), surcharge percentage (optional).
- Promo codes: code, discount type (% or fixed), value, max uses, expiry, applicable formations.

Codebase touch points

- supabase/migrations/<timestamp>_promo_codes.sql.
- src/app/admin/pricing/* — UI.
- src/app/api/admin/pricing/* — endpoints that push config to Stripe via API.

Important notes

- Stripe Products / Prices are SOURCE OF TRUTH. Admin pushes config TO Stripe — do not maintain a parallel pricing database that can drift.
- Pricing changes do NOT affect existing customers.

✓ **Acceptance criteria**

- Changing Full Body price syncs to Stripe within 30 seconds and reflects on the public purchase page.
- Enabling installments adds the option to the public checkout without dev intervention.
- Content Manager sees pricing fields as read-only.
- Promo code expires automatically at midnight on expiry date.

Manual test (how to prove it works)

As Super Admin, change Full Body price from €299 to €349. Refresh the public purchase page within 60s — new price is visible. Open Stripe dashboard — price is updated there too.

Part E — Governance & admin team

T20–T21 handle who can do what, and what they did.

TICKET #20 — Audit log — append-only event capture

Type: **ADD** | Priority: **HIGH** | Section: Governance / Audit

Starting point: **NOT BUILT — net new**

TL;DR

Every admin mutation produces an immutable log entry: who, when, what changed (before/after). Super Admin only.

Why this matters

Two-team operation needs accountability. When something breaks or a user complains, the audit log is how to reconstruct what happened. Append-only is non-negotiable — the value of a log that can be edited is zero.

What exists today

- No audit logging anywhere.

What needs to be built

Append-only table + viewer UI (Super Admin only).

Use Common Pattern 2 to wrap every mutation route.

Required capabilities

- Table `audit_log`: `id`, `admin_id`, `admin_email_snapshot`, `admin_role_snapshot`, `action_type`, `entity_type`, `entity_id`, `before_json`, `after_json`, `ip_address`, `user_agent`, `created_at`. APPEND-ONLY (revoke UPDATE/DELETE at the DB level).
- Capture: user profile edit, manual access grant/revoke, progress override, refund issued, formation/module/video/quiz/resource/testimonial create/edit/archive, CMS edit, legal disclaimer edit, attestation template edit, admin invite/remove/role-change, pricing change, promo code create, login failures.
- Filterable by admin, action type, date range, entity type.
- Export to CSV for compliance.
- Snapshot admin email + role at write time so deleted admins still have intelligible entries.

Codebase touch points

- `supabase/migrations/<timestamp>_audit_log.sql` — table with INSERT-only RLS for `service_role`.
- `src/lib/audit.ts` — single helper called from every mutation route.
- `src/app/admin/audit-log/page.tsx` — viewer UI.
- `src/app/api/admin/audit-log/route.ts` — read endpoint.

Important notes

- Implement as middleware or wrapper around mutation routes so it cannot be accidentally skipped.
- `before_json` / `after_json` should capture only the specific changed fields, not the entire entity blob (signal-to-noise).

✓ Acceptance criteria

- Every action above produces an audit log entry within 5 seconds.
- Audit log entries cannot be modified or deleted via UI or API (verify by attempting both).
- Filtering by admin email returns only that admin's actions.
- Deactivated admins' historical entries remain readable with snapshot email + role.

Manual test (how to prove it works)

As Content Manager, edit a test user's phone number. Open audit log as Super Admin — entry exists with before/after phone values. Try DELETE via SQL on the audit_log table — DB returns permission denied.

TICKET #21 — Admin team — invite, role change, deactivate

Type: **ADD** | Priority: **HIGH** | Section: Governance / Team

Starting point: **NOT BUILT — net new**

TL;DR

Super Admins invite new admins, change roles, deactivate accounts, force-logout. Cannot deactivate the last Super Admin.

Why this matters

Without this, every new METCARE® team member or Agentum hire = a developer running a DB insert. Also a security floor: deactivation must invalidate sessions immediately.

What exists today

- Only one admin tier exists.
- Admin creation is manual (DB insert / Supabase dashboard).

What needs to be built

Super Admin only — sidebar item hidden for Content Managers.

Invite, list, change role, deactivate, force-logout.

Required capabilities

- Admin list: name, email, role, status (active|pending|deactivated), last login.
- Invite: email, name, role, optional welcome note. Resend email with a one-time signup link (valid 7 days).
- Change role (super_admin ⇌ content_manager). Captured in audit log.
- Deactivate (not delete). Deactivated admins cannot log in; audit log history preserved.
- Force logout: invalidates all of an admin's sessions immediately.
- Cannot deactivate the last remaining Super Admin (prevent lockout).

Codebase touch points

- src/app/admin/team/* — UI.
- src/app/api/admin/team/route.ts — invite, list.
- src/app/api/admin/team/[id]/route.ts — role change, deactivate.
- src/app/api/admin/team/[id]/force-logout/route.ts — session invalidation.

✓ Acceptance criteria

- Invited admin receives the email within 60 seconds.
- Invite link expires after 7 days and cannot be reused.
- Deactivating invalidates target admin's sessions within 60 seconds.
- System refuses to deactivate the last Super Admin with a clear error.
- Role changes capture previous + new role in audit log.



Manual test (how to prove it works)

Invite a new Content Manager. They receive the email, sign up, enroll 2FA, reach the dashboard. As the only existing Super Admin, try to deactivate yourself — system blocks it with 'cannot deactivate last Super Admin'.

Part F — System settings

T22 covers the ops knobs: support widget, brand colours, notification recipients, in-app announcement banner, maintenance mode.

TICKET #22 — System settings — support, branding, banner, notifications

Type: **ADD** | Priority: **MEDIUM** | Section: System / Settings

Starting point: **NOT BUILT — net new**

TL;DR

Single settings page covering: support email + widget URL, notification recipients, brand colours, in-app announcement banner, maintenance mode.

Why this matters

Several requirements from the e-learning brief don't fit neatly into content or users — the support widget (T30), the `contact@myestheticttravel.com` notification recipient (T06), encouragement-message tone. Plus general ops needs (banners, maintenance mode). Without this, each is a hard-coded value requiring dev help to change.

Maps to: *E-learning brief T06 (notification recipients), T30 (support widget).*

What exists today

- No system settings page exists.

What needs to be built

Single settings page organized into sections.

Support: support email address, support widget URL (or 'none').

Required capabilities

- **SUPPORT** section: `support_email` (shown on every public page footer), `support_widget_url` (optional Intercom/Crisp/Tawk URL — embedded site-wide if set), `support_help_icon_visible` (bool, default true).
- **NOTIFICATION** section: `purchase_notification_emails` (list of recipient emails — replaces hard-coded '`contact@myestheticttravel.com`' for purchase CC per e-learning T06), `refund_notification_emails` (list), `critical_error_emails` (list).
- **BRANDING** section: brand primary colour, brand secondary colour, logo upload, favicon upload, public-site default language (FR|EN). Reflected on public site via CSS variables.
- **ANNOUNCEMENT BANNER**: title, message, link (optional), `display_until` (date), visibility (visitors|free_users|paid_users|all), background colour, dismissible (bool). When active, shows site-wide banner above the header.
- **MAINTENANCE MODE**: toggle that shows a maintenance page to all non-admin users. Custom message FR/EN. Admin portal continues to work normally.
- **Permission**: Super Admin only for branding, maintenance mode, and notification recipients. Content Managers can edit support email + announcement banner.

Codebase touch points

- supabase/migrations/<timestamp>_system_settings.sql — single-row settings table (or KV-style table).
- src/app/admin/settings/* — tabbed UI per section.
- src/app/api/admin/settings/* — endpoints.
- Public site — read settings via cached API, expose support widget, banner, maintenance check.

Important notes

- Maintenance mode is a kill switch — bake in a hard test before shipping (admin tries to enable, then disable, on a staging environment).
- Brand colours should sync to public-site CSS variables on save via the revalidation pattern.

Acceptance criteria

- Changing the support email reflects on the public site footer within 60 seconds.
- Adding an announcement banner displays on the public site within 60 seconds.
- Enabling maintenance mode shows the maintenance page to all non-admin visitors; admin portal continues to work.
- Content Manager attempting to edit brand colours sees fields as read-only.



Manual test (how to prove it works)

Enable announcement banner with message 'Nouvelle formation disponible' and a link. Refresh public site — banner displays at the top with the message. Dismiss it in the public site UI — it disappears for that visitor. Disable maintenance mode toggle is on; refresh public site as anon visitor — maintenance page appears. Refresh /admin — admin portal works normally.



Appendix A — Data model

Stack-locked: Supabase PostgreSQL. Below is the recommended schema. The relationships are non-negotiable.

New tables (Phase 2)

- admins — id, email (unique), name, password_hash, role, totp_secret, totp_enrolled, status, last_login_at, failed_login_count, locked_until, created_at.
- formations — id, slug, name_fr, name_en, description_fr, description_en, price_cents, currency, requires_initiation_id (nullable self-FK), is_free, formation_type, status, display_order.
- videos — id, module_id (FK), title_fr, title_en, description_fr, description_en, cloudflare_stream_id, duration_seconds, thumbnail_url, display_order, status.
- quizzes — id, title_fr, title_en, module_id (FK), pass_threshold_pct, max_attempts (nullable), shuffle_questions, shuffle_answers, status.
- questions — id, quiz_id (FK), text_fr, text_en, display_order, explanation_fr, explanation_en.
- answers — id, question_id (FK), text_fr, text_en, is_correct, display_order.
- quiz_attempts — id, user_id (FK), quiz_id (FK), score_pct, passed, attempted_at.
- resources — id, type (audio|document), title_fr, title_en, description_fr, description_en, file_url, thumbnail_url, access_formation_id (FK), display_order, status, download_count, play_count.
- testimonials — id, author_name, author_role, country, photo_url, quote_fr, quote_en, rating, display_order, status, tagged_formation_id (nullable).
- page_content — id, page_key, field_key, value_fr, value_en, updated_by_admin_id, updated_at. UNIQUE (page_key, field_key).
- page_content_revision — id, page_content_id (FK), value_fr, value_en, admin_id, created_at.
- attestation_templates — id, formation_id (FK), heading_fr, body_fr, logo_url, signature_image_url, signature_name, updated_at.
- user_formation_access — id, user_id (FK), formation_id (FK), source (purchase|manual_grant|promo), granted_by_admin_id (nullable), reason, stripe_charge_id (nullable), granted_at, revoked_at (nullable), revoked_by_admin_id (nullable), revoke_reason.
- user_progress_overrides — id, user_id (FK), video_id (FK), override_type (mark_complete|mark_incomplete|reset), override_reason, admin_id (FK), created_at.
- transactions — id, user_id (FK), formation_id (FK), stripe_payment_intent_id (unique), amount_cents, currency, status, payment_method, refunded_amount_cents, created_at.
- promo_codes — id, code (unique), discount_type (percent|fixed), discount_value, max_uses, uses_count, applicable_formation_ids (array), expires_at.
- audit_log — id, admin_id, admin_email_snapshot, admin_role_snapshot, action_type, entity_type, entity_id, before_json, after_json, ip_address, user_agent, created_at. APPEND-ONLY.
- system_settings — single-row or KV-style table holding support email, widget URL, notification recipients, brand colours, banner config, maintenance mode.

Modified tables (Phase 1 → Phase 2)

- users — add (if not present): phone, profession, country, main_objective. Existing status column stays.
- modules — add: formation_id (FK NOT NULL after backfill), description_fr, description_en, completion_required, quiz_id (nullable FK), has_quiz, is_free (for marking modules free within a paid formation).

Critical relationships & invariants

- Formation self-reference via requires_initiation_id is how the tronc commun is enforced. Never hard-code 'Initiation' in app code.
- user_formation_access.source separates purchase from manual grant — this is what makes refund and revenue reporting accurate.
- audit_log snapshots admin email + role at write time — deleted/deactivated admins still produce intelligible entries.
- page_content_revision retains last 10 versions per (page_key, field_key) — older revisions pruned by a scheduled job.
- attestation_templates content fields are server-side-validated to reject 'certification' / 'certifié' / 'certifie' (case-insensitive). Legal requirement per e-learning T14.
- user_progress_overrides creates an audit trail for any manual progress mutation — never edit user_progress rows directly.

Appendix B — New environment variables

Additions to the existing Phase 1 env var list:

Variable	Used in	Purpose
ADMIN_SESSION_SECRET	Admin Portal	Signs/verifies admin session tokens. Distinct from JWT_STUDENT_SECRET.
ADMIN_TOTP_ENC_KEY	Admin Portal	Encrypts TOTP secrets at rest in the admins table.
STRIPE_SECRET_KEY	Admin Portal + Backend	Stripe API for charges and refunds.
STRIPE_WEBHOOK_SECRET	Admin Portal	Verifies incoming Stripe webhook signatures.
RESOURCES_AUDIO_BUCKET	Admin Portal	Supabase storage bucket for Pro guide audio.
RESOURCES_DOCS_BUCKET	Admin Portal	Supabase storage bucket for Pro guide PDFs.
PUBLIC_SITE_REVALIDATE_URL	Admin Portal	Public-site endpoint triggered when CMS content changes.
PUBLIC_SITE_REVALIDATE_TOKEN	Admin Portal	Shared secret for the revalidate endpoint.

Final checklist for developer

Before marking Phase 2 complete, verify ALL of:

Foundation

- T00 migration ran on production without data loss; existing modules grouped under 'Full Body' formation.
- T01 admin login fully separated from student login — verified by attempting cross-login both ways.
- T01 2FA mandatory and tested on first login.
- T01 role-based access enforced server-side, not just hidden in UI (test by typing /admin/audit-log URL as Content Manager).

Content

- T03 adding a new formation requires zero code changes.
- T05 bulk upload of 21 Full Body videos completes in one admin session.
- T09 homepage copy edits publish to live site within 60 seconds via revalidation.
- T09 covers ALL of: public pages + dashboards + lock messages + milestone messages + encouragement messages.
- T11 attestation template editor rejects 'certification' (case-insensitive).

Users

- T12 user search returns results in under 1 second across 10,000+ records.
- T14 manual access grants flow through the same unlock cascade as paid purchases.
- T15 per-user progress override is captured in audit log with required reason.
- T16 CSV exports are UTF-8 with BOM — French characters intact in Excel.

Payments

- T17 Stripe webhook receiver writes to mirror table within 60s; figures match Stripe within €1.
- T18 refunds revoke access atomically — no partial states possible.
- T19 Stripe pricing changes sync via API; admin panel does not maintain parallel pricing.

Governance & system

- T20 every significant action produces an audit log entry within 5 seconds.
- T20 audit log entries cannot be modified or deleted by anyone (verify by attempting both).
- T21 cannot deactivate the last remaining Super Admin.
- T22 maintenance mode tested on staging — enables, disables, admin portal continues to work.

Regression checks (Phase 1 still works)

- User activate / disable buttons in UsersTab.tsx continue to function.
- Module thumbnail upload continues to function (race condition fix preserved).
- Student-side deactivation kick (studentAuth + global interceptor) continues to function.
- EN/FR i18n keys exist for every new admin UI string.

- Per Phase 1 'Known remaining items': frontend vercel.json + backend FRONTEND_URL env confirmed on Vercel.

— *End of Brief* —

Prepared by Ali Masood — Agentum AI
For METCARE® / MyEstheticTravel — MET Academy Admin Portal Phase 2